Planning with Motion Limits

Contents

1. Overview 2
2. Speed is part of the state: phase space 4
   - 2.1 Why position is not enough 4
   - 2.2 The state, its motion equations and the stopping distance 5
   - 2.3 Kinodynamic planning 7
3. States that are already lost: inevitable collision states 7
   - 3.1 Obstacles in phase space 8
   - 3.2 The region of inevitable collision 8
4. Where the robot can get to: reachable sets 10
5. A few motions to build with: motion primitives and a system simulator 11
6. Searching the primitives systematically: lattice search 14
7. The shortest drivable paths: Dubins and Reeds-Shepp curves 17
   - 7.1 Dubins paths: forward only 17
   - 7.2 Reeds-Shepp paths: with reverse 19
   - 7.3 Using them in a planner: a drivable steer 20
8. Growing a tree of drivable motions: kinodynamic RRT 21
9. Summary 24
10. Sources 25
11. Key terms 26

Where this fits:

- **Builds on:** Differential-drive kinematics; Nonholonomic constraints; Breadth-first search; Configuration space; Obstacle region and free space; Rapidly-exploring random tree (RRT).
- **Leads to:** Hybrid A*; State lattice and control set; Trajectory optimisation.
- **Compare with:** Trajectory optimisation.

1. Overview

Key point: The planners so far assume the robot can stop dead and turn on the spot at any moment, but at speed it can do neither: from 0.6 m/s it needs 0.36 m to stop, and at 0.5 m/s its tightest turn is a circle of 0.5 m. Planning with motion limits puts the speed into the state and builds every path from motions the robot can really drive.

The differential-drive robot of RO-001, at (2, 1) facing 30° in the room, already has several ways to plan a route to the door:

- A* (G-2577) on the 0.5 m grid gives a 3.414 m route of straight moves with 45° corners;
- the drive checked in C-space turns on the spot to 61.9° and drives a straight 3.4 m to the doorway;
- the rapidly-exploring random tree (RRT) (G-2761) grows straight edges towards random samples.

All three join straight pieces at corners, and each assumes the robot can stop at any point and turn there on the spot. The assumption held while we only asked where the robot could be. The assumption breaks once the robot moves at a useful speed, because its motors have limits. We give our robot two:

- each wheel turns at most fast enough for 0.6 m/s over the floor;
- the forward speed changes by at most 0.5 m/s every second, the acceleration limit.

Figure 1 plays what the limits do to two of these plans. Watch the left panel first: the plan drives at the top speed, 0.6 m/s, straight at the doorway, whose door is now closed, and stops at 3.0 m along the line. The robot cannot stop in an instant, so it brakes from 3.0 m and still meets the door at 0.49 m/s. Then the right panel: at 0.5 m/s the robot reaches the first 45° corner of A*'s route and turns as fast as it can, 1 rad/s; it swings 0.146 m to the right of the route, onto floor the route never checked.

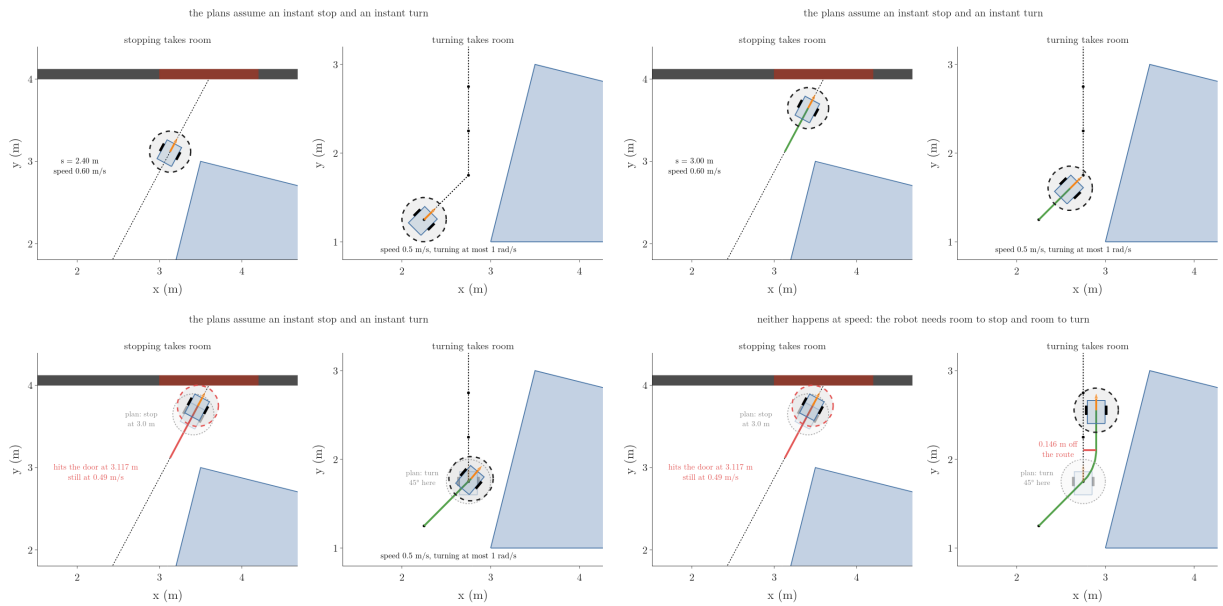


Figure 1: Two plans driven with the robot's real limits. Left, stopping: on the straight line to the closed door, with s the distance along the line, the plan cruises at 0.6 m/s and stops at s = 3.0 m (pale robot); braking at 0.5 m/s^2 the robot reaches the door at s = 3.117 m still at 0.49 m/s (red). Right, turning: on A*'s route at 0.5 m/s the plan turns 45° at the corner (2.75, 1.75) on the spot (pale robot); turning at its limit of 1 rad/s the robot swings 0.146 m to the right of the route. The last frame holds both outcomes

Where do 1 rad/s and the 0.5 m circle come from? By the inverse kinematics (G-2295) of RO-001, the outer wheel runs faster than the robot by the turn rate times half the wheel separation. At 0.5 m/s only 0.1 m/s of wheel speed is left for turning. With:

- v, the robot's linear velocity (G-2291), 0.5 m/s;
- ω, its angular velocity (G-2292), the turn rate, in rad/s;
- L, the wheel separation, 0.2 m;
- $v_{\max}$, the top wheel speed, 0.6 m/s;
- R, the turning radius (G-2293), in m.

The outer wheel at its top speed, our numbers first, then the same steps in symbols:

$$0.5 + \omega \times 0.1 = 0.6$$

$$v + \omega L/2 = v_{\max}$$

$$\omega = 0.1/0.1 = 1 \text{ rad/s}$$

$$\omega = 2\,(v_{\max} - v)/L$$

$$R = 0.5/1 = 0.5 \text{ m}$$

$$R = v/\omega$$

The numbers are RO-001's: wheels at 0.6 and 0.4 m/s give 0.5 m/s, 1 rad/s and a circle of 0.5 m. With the 0.6 m/s limit, that circle is now the tightest one the robot can drive at 0.5 m/s.

The opening question of this Note: **how does the robot plan a drive to the door that it can follow at speed, stopping in time and turning only as tightly as its wheels allow?** We answer it in two parts:

- **stopping:** put the speed into what the planner tracks (Section 2), find the states from which a crash can no longer be avoided (Section 3), and search the fastest safe drive (Section 6);
- **turning:** find where the robot can get to in a short time (Section 4), build paths from a few drivable motions (Section 5), learn the shortest drivable paths (Section 7), and grow an RRT out of drivable motions (Section 8).

2. Speed is part of the state: phase space

> **Key point:** Whether the robot can still stop before the door depends on its position and its speed together, so the planner must track both. The pair (position, speed) is a point of the **phase space** (G-2820); planning there with the limits on how the speed can change is **kinodynamic planning** (G-2821).

2.1 Why position is not enough

The stopping part of the opening problem starts with a simple question: at s = 2.8 m along the line to the closed door, is the robot safe? Figure 2 answers it with two robots at that same place, both braking as hard as they can. Watch robot A at 0.3 m/s stop at 2.89 m. Robot B, at 0.6 m/s, is at the same place but meets the door at 3.117 m still moving at 0.21 m/s.

The right panel of Figure 2 draws the same two runs in a new picture. Position along the line goes across and speed goes up, so each robot at each moment is one point. Robot A starts at (2.8 m, 0.3 m/s) and robot B at (2.8 m, 0.6 m/s). Braking moves each point down, because the speed falls, and to the right, because the robot still moves forward. A's point reaches the speed 0 before the door; B's point reaches the door, the dark red band, above 0.

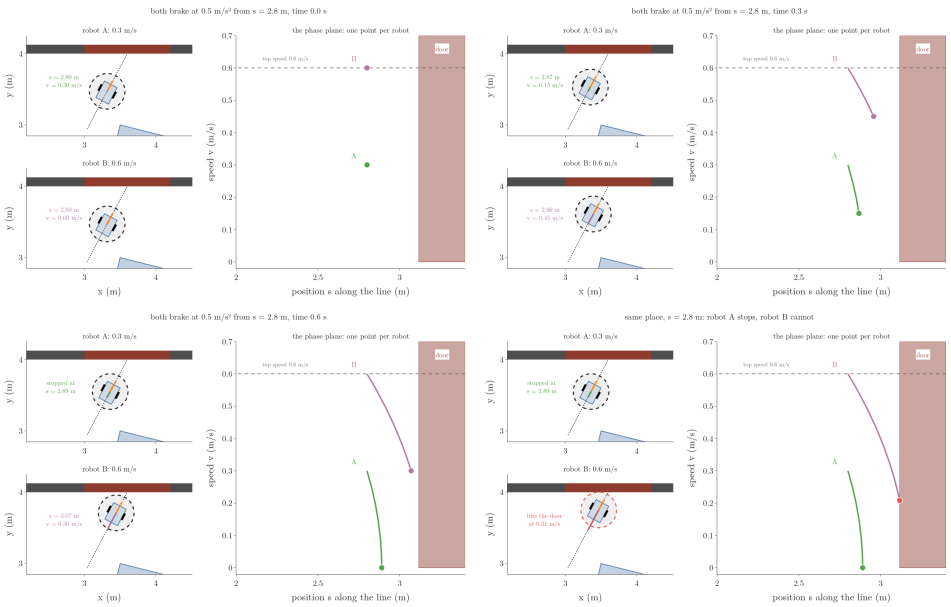


Figure 2: Two robots at the same place, s = 2.8 m along the line to the closed door (s, the distance from the start), brake at 0.5 m/s^2 from time 0. Left: robot A at 0.3 m/s (green) stops at s = 2.89 m; robot B at 0.6 m/s (purple) reaches the door at s = 3.117 m still at 0.21 m/s (red). Right: the same runs as points in the plane of position s (across, 2.0 to 3.4 m) and speed v (up); braking moves each point down and to the right along a curve; the dark red band is the door, every s from 3.117 m on. The last frame holds both outcomes

The same position gives two different futures. So the planner must keep the speed as well as the position: the two numbers together decide what the robot can still do, in the same way that the heading (G-2284) joined the position in RO-001.

2.2 The state, its motion equations and the stopping distance

For the drive along the line we track two numbers, and the robot changes them through one input:

- s, the position along the line from the start (2, 1), in m;
- v, the speed along the line, in m/s, from 0 to 0.6 m/s;
- u, the acceleration the robot commands, in m/s^2, from -0.5 to 0.5.

The pair (s, v) is the state of the drive. A space of configurations and their speeds, such as these pairs, is the **phase space** (G-2820) (Hauser ch.11; LaValle §14.1.3). The picture on the right of Figure 2 is the phase space of the straight drive, also called the phase plane because it has two numbers.

Speed grows at the rate u, and position at the rate v. In the dot notation (G-2296) of RO-001, where a dot means "rate of change per second":

$$\dot{s} = v$$

$$\dot{v} = u$$

Equations like these, which give the rate of change of each number of the state from the state and the input, are called differential equations. For any robot we write them in one line (Hauser ch.11; MR §10.5), with:

- $\mathbf{x}$, the state, such as (s, v), in bold so that it is not confused with the x coordinate;
- u, the input, such as the acceleration;
- f, the function that gives the rates from the state and the input, here (v, u).

$$\dot{\mathbf{x}} = f(\mathbf{x}, u)$$

The stopping distance. How much room does braking need? Braking at the full 0.5 m/s^2, the speed falls steadily to 0, so its average over the stop is half the starting speed. For robot B:

- v, the speed when braking starts, 0.6 m/s;
- a, the braking rate, the acceleration limit, 0.5 m/s^2;
- t, the time to stop, in s;
- d, the distance travelled while stopping, in m.

Our numbers first, then the same steps in symbols:

$$t = 0.6/0.5 = 1.2 \text{ s}$$

$$t = v/a$$

$$d = (0.6/2) \times 1.2$$

$$d = (v/2) \times t$$

$$d = 0.3 \times 1.2 = 0.36 \text{ m}$$

$$d = v^2/(2a)$$

Robot B has this much room left before the door:

$$3.117 - 2.8 = 0.317 \text{ m}$$

less than the 0.36 m it needs (notebook). Robot A, at 0.3 m/s, needs only 0.09 m of the same 0.317 m.

2.3 Kinodynamic planning

The planners of chapter RO-04 searched the configuration space (G-2287): they asked only where the robot could be, and assumed it could move between any two nearby configurations. With motion limits, the planner searches the **state space** (G-2545), all the states the problem can be in, here configurations with their speeds, and may only use motions that the motion equations produce with allowed inputs. Planning a path that respects both the obstacles and these limits on how the state can change is **kinodynamic planning** (G-2821) (Hauser ch.11). The name was first used for planning with bounds on velocity and acceleration, as on our straight drive, and now covers any planning with such limits on how the state changes (LaValle §14.1.2.1).

Formally, with:

- $\mathbf{x}(t)$, the state at time t, such as (s, v) for the straight drive;
- $u(t)$, the input at time t, inside its allowed set, such as u between -0.5 and 0.5 m/s^2;
- X_{free}, the states that touch no obstacle;
- $\mathbf{x}_I$, the start state, such as rest at the start: (0 m, 0 m/s);
- X_G, the set of goal states, such as rest in front of the door;
- T, the time the drive takes, in s.

a kinodynamic plan is a pair of curves $\mathbf{x}(t)$ and $u(t)$, for t from 0 to T, such that the four lines below hold (Hauser ch.11). The sign $\in$ reads "is in"; for example, robot A's state, (2.8 m, 0.3 m/s), is in X_{free}.

$$\mathbf{x}(t) \in X_{\text{free}} \text{ for every } t$$

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), u(t))$$

$$\mathbf{x}(0) = \mathbf{x}_I$$

$$\mathbf{x}(T) \in X_G$$

The first line keeps the robot off obstacles; the second allows only motions the robot can make. The rest of the Note builds such plans. First we need to know which states X_{free} really leaves out: robot B at 2.8 m touches nothing yet, but it is already lost.

3. States that are already lost: inevitable collision states

> **Key point:** Some states touch no obstacle yet cannot avoid one, because every input leads into the door; such states form the **region of inevitable collision** (G-2823). On the straight drive the region holds every state whose stopping distance is longer than the room left, so it grows with the square of the speed.

3.1 Obstacles in phase space

Robot B of Figure 2 hit the door from a state that looked free. To see why, we first mark the states that are plainly forbidden. The door stops the disk at s = 3.117 m, whatever the speed: the obstacle region (G-2650) of the line is every s from 3.117 m on. In the phase plane it becomes the band of every state (s, v) with that s and any speed v, the dark red band of Figures 2 and 3. The set of states whose configuration touches an obstacle, at any speed, is the **obstacle region in phase space** (G-2822) (LaValle §14.1.3). The top speed adds a second kind of limit: no state above 0.6 m/s is allowed (LaValle §14.1.3.1).

3.2 The region of inevitable collision

The band is not the whole danger. A state left of the band is lost if the robot needs more room to stop than it has. Braking hardest is its best try, because any other input leaves it faster and closer to the door. With:

- s, the position along the line, in m;
- v, the speed, in m/s;
- a, the braking rate, 0.5 m/s^2;
- s_{door}, where the disk meets the door, 3.117 m.

the state is lost when the stopping distance of Section 2.2 is longer than the room left:

$$\frac{v^2}{2a} > s_{\text{door}} - s$$

Robot B, at 2.8 m and 0.6 m/s:

$$\frac{0.6^2}{2 \times 0.5} = 0.36$$

$$3.117 - 2.8 = 0.317$$

$$0.36 > 0.317$$

so B is lost. Robot A, at 0.3 m/s, needs 0.09 m of its 0.317 m, so A is safe. Figure 3 shades every lost state orange. Look at its left edge: it is the braking curve that just touches the door. Read the widths across it: the shaded strip is 0.04 m wide at 0.2 m/s, 0.16 m at 0.4 m/s and 0.36 m at 0.6 m/s. Twice the speed needs four times the room.

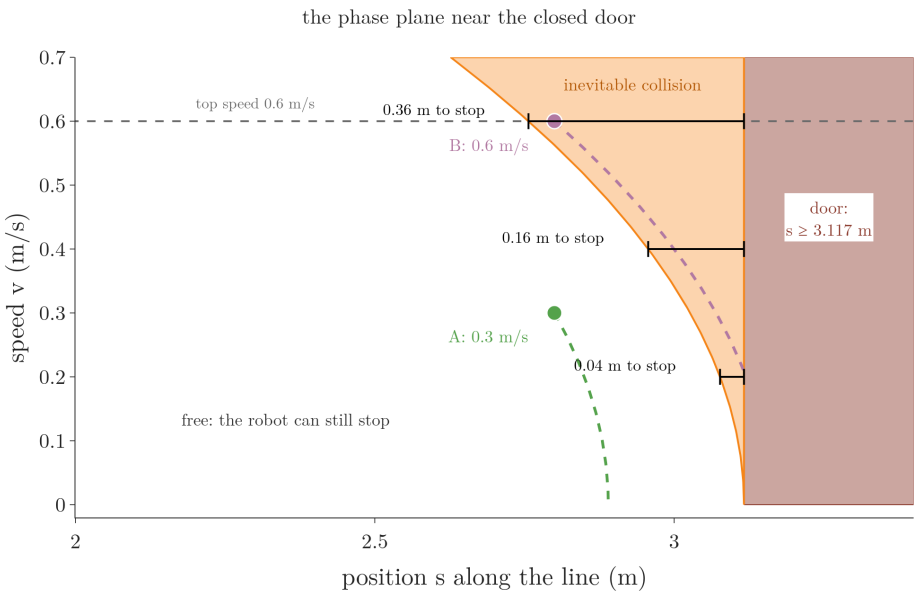


Figure 3: The phase plane of the straight drive near the closed door: position s across (2.0 to 3.4 m), speed v up (0 to 0.7 m/s). Dark red: the door, every state with s of at least 3.117 m. Orange: the region of inevitable collision, every state whose stopping distance is longer than the room left; its left edge is the braking curve that ends exactly at the door. The black bars give the stopping distance at three speeds: 0.04 m at 0.2 m/s, 0.16 m at 0.4 m/s, 0.36 m at 0.6 m/s. Robot A (2.8 m, 0.3 m/s, green) lies outside and its braking curve (dashed) ends at rest; robot B (2.8 m, 0.6 m/s, purple) lies inside and its braking curve reaches the door. Grey dashed line: the top speed, 0.6 m/s

The states from which every input sooner or later leads into the obstacle region form the **region of inevitable collision** (G-2823); its states are called inevitable collision states (LaValle §14.1.3.2). LaValle's example: a vehicle at 100 km/h only 2 m from a wall is doomed whatever it does, although it touches nothing yet. For a robot that moves only along a line, with acceleration at most a, the region grows quadratically with the speed because the stopping distance does (LaValle Example 14.1). At the top speed of 0.6 m/s the robot must start braking by:

$$3.117 - 0.36 = 2.757 \text{ m}$$

and at the cruising speed of 0.5 m/s by 2.867 m (notebook). The 2.757 m mark answers the first half of the stopping part: braking from 3.0 m at 0.6 m/s in Figure 1 started 0.243 m too late.

A planner that only checks for collisions with the obstacle region can wander into such states and discover the crash only later. In two or three dimensions the region also depends on where the robot can swerve, and no practical test for it is known; sampling planners work with the obstacle region and let the momentum show up as dead ends (LaValle §14.1.3.2).

On our line the test is exact, and Section 6 uses it to find the fastest safe drive. Before that, the turning part: where can a robot that keeps moving get to at all?

4. Where the robot can get to: reachable sets

> **Key point:** The **reachable set** (G-2824) of a state is every state the robot can reach from it within a time limit, using allowed inputs. At 0.5 m/s with turns of at most 1 rad/s it is a fan in front of the robot, and the plans' "turn on the spot" pose is never inside it.

The turning part of the opening problem: the earlier plans turn the robot on the spot, and at 0.5 m/s it cannot. Which poses can it reach instead? We let the robot start at (2, 1) facing 30°, as in Figure 1, driving at 0.5 m/s, and allow any turn rate between −1 and 1 rad/s, changed whenever we like. Figure 4 draws 403 such runs as time T grows from 0 to 1.5 s. Watch the green trails spread into a fan. Its two edges are the tightest left and right circles, dotted, of radius 0.5 m; nothing outside them can be reached.

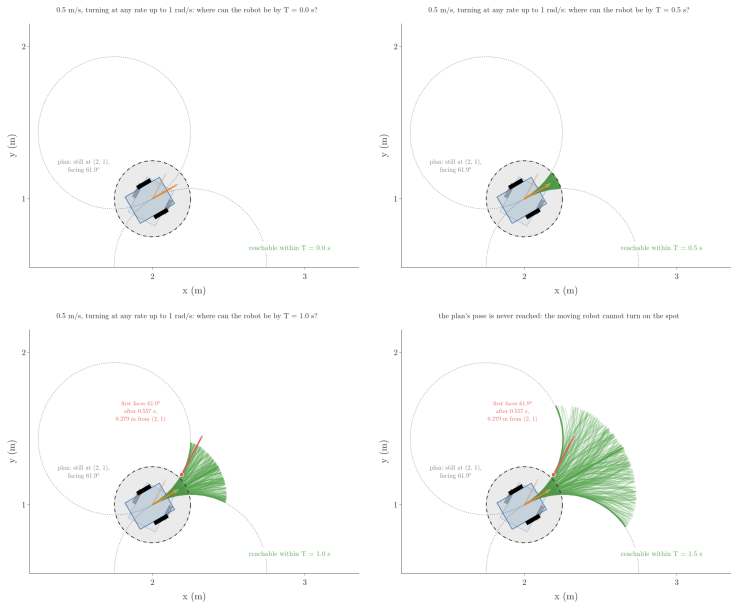


Figure 4: Positions the robot can reach within T seconds when it starts at (2, 1) facing 30° at 0.5 m/s and turns at any rate between −1 and 1 rad/s: 400 runs with random turn rates held for random times (pale green) and the full left, straight and full right runs (dark green), drawn up to T, which grows from 0 to 1.5 s. The dotted circles of radius 0.5 m are the tightest left and right turns. The pale robot is the plan's pose: still at (2, 1) but facing 61.9°. The red arrow marks the first moment the robot can face 61.9°: after 0.557 s, 0.279 m from (2, 1), at (2.191, 1.198)

The plan's pose of Section 1, the robot still at (2, 1) but facing the doorway at 61.9°, never enters the fan. Turning the 31.9° that the plan asks for, 0.557 rad, takes 0.557 s at 1 rad/s,

and in that time the robot drives 0.279 m. So the earliest moment it faces 61.9° is at (2.191, 1.198), not at (2, 1) (notebook).

The positions the robot can reach from a state within a time T, with every allowed input, form the **reachable set** (G-2824) of that state, also called the time-limited reachable set (LaValle §14.2.1; MR §13.3.2). With:

- $\mathbf{x}_0$, the starting state, such as (2, 1, 30°) at 0.5 m/s;
- $\tilde{u}$, an allowed input curve, here any turn rate between −1 and 1 rad/s at each moment;
- T, the time limit, such as 1.5 s in Figure 4;
- $\mathbf{x}(t')$, the state the input curve leads to at time t', no later than T;
- $\mathbf{x}_1$, a state of the set;
- the braces {}, read "the set of", and the bar |, read "such that".

the reachable set within time T is (LaValle §14.2.1.2, eq. 14.5):

$$R(\mathbf{x}_0, T) = \{\mathbf{x}_1 \mid \mathbf{x}_1 = \mathbf{x}(t')$$

$$\text{for some } \tilde{u} \text{ and } t' \leq T\}$$

In words: every state $\mathbf{x}_1$ that some allowed input reaches at some time no later than T. Figure 4 draws only its positions; the full set also holds a heading for each position.

The shape of the fan explains the planners' trouble. A point 0.3 m to the robot's left is near in distance but not in the fan at all: to reach it the robot must drive a loop. With a reverse gear, a car can also reach points beside it by shuffling forward and back, as in parallel parking, but even then it cannot move straight sideways (MR §13.3.2; LaValle §14.2.1.2). So a planner must build paths only from motions that stay inside such fans. The next section picks a few of them.

5. A few motions to build with: motion primitives and a system simulator

> **Key point:** Instead of all turn rates, a planner uses a few: full left, straight and full right, each held for 0.75 s. Each such motion is a **motion primitive** (G-2826), and the **system simulator** (G-2825) works out where it takes the robot by stepping the motion equations forward.

Figure 4 used every turn rate from −1 to 1 rad/s. A planner cannot try infinitely many, so it picks a few. Which ones? Figure 5 shows where the choice comes from. All the pairs of forward speed and turn rate the robot can produce, with each wheel at most 0.6 m/s, form a diamond: full speed at its left and right corners, and spinning on the spot, 6 rad/s, at its top and bottom (MR §13.3.1, Figure 13.11). At the cruising speed of 0.5 m/s only the green segment is left: turn rates from −1 to 1 rad/s. We take its two ends and its middle.

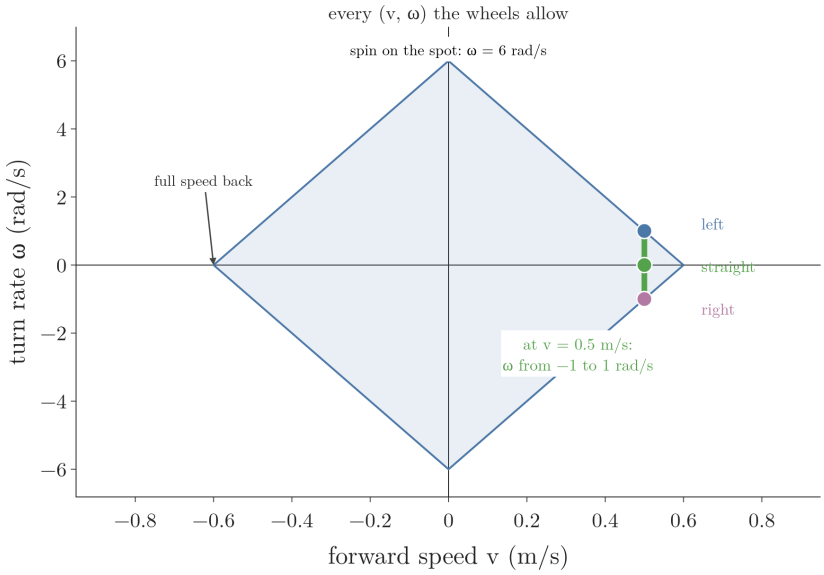


Figure 5: Every pair of forward speed v (across) and turn rate omega (up) it can produce with each wheel at most 0.6 m/s: a diamond with corners at v = ±0.6 m/s and omega = ±6 rad/s (spinning on the spot). At the cruising speed v = 0.5 m/s only turn rates from −1 to 1 rad/s are inside (green segment); the three primitives use its ends and its middle: left (1 rad/s), straight (0) and right (−1 rad/s)

Holding one input for a fixed time gives a short drivable motion. Each such piece is a **motion primitive** (G-2826) (LaValle §14.2.3; MR §10.5). Our three primitives hold 0.5 m/s with a turn rate of 1, 0 or −1 rad/s for 0.75 s, so each drives 0.375 m and turns by 0.75 rad (43°), 0 or −0.75 rad. The full left, straight and full right pieces are written L, S and R (MR §13.3.3). More general primitives may last different times, such as a recorded parking manoeuvre (LaValle §14.2.3).

Where does a primitive end? The motion equations give only rates, so we add up small steps: the Euler steps (G-2299) of RO-001, repeated. The module that does this, given a state, an input and a duration, is the **system simulator** (G-2825) (LaValle §14.3.2; Hauser ch.11). A planner treats it as a black box, the way it treats the collision checker: it asks "where does this input take me?" and gets back the states along the way.

The first Euler step of the left primitive, with the kinematic model (G-2297) of RO-001:

- x, y, θ, the pose at the start, (2, 1, 0.5236 rad), which is 30°;
- v, 0.5 m/s;
- ω, 1 rad/s;
- Δt, the step, 0.01 s.

$$x_{\text{new}} = 2 + 0.5 \times 0.866 \times 0.01$$

$$= 2.00433$$

$$y_{\text{new}} = 1 + 0.5 \times 0.5 \times 0.01$$

$$= 1.0025$$

$$\theta_{\text{new}} = 0.5236 + 1 \times 0.01$$

$$= 0.5336$$

Figure 6 runs the simulator on all three primitives, 75 steps each. Watch the dots, one every 5 steps: the left primitive curls up to 73°, the straight one keeps 30°, the right one bends down to −13°.

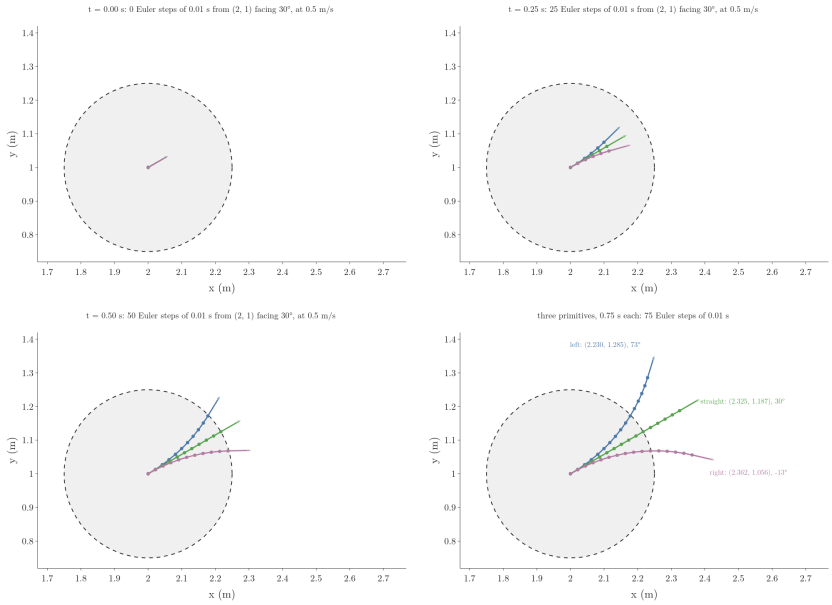


Figure 6: The system simulator builds the three primitives from (2, 1) facing 30° at 0.5 m/s: Euler steps of 0.01 s, a dot every 5 steps (0.025 m), 75 steps in all. Last frame: left ends at (2.230, 1.285) facing 73°, straight at (2.325, 1.187) facing 30°, right at (2.362, 1.056) facing −13°

The three end poses are (2.230, 1.285) at 73°, (2.325, 1.187) at 30° and (2.362, 1.056) at −13° (notebook). The exact circle gives the same to within 2 mm. A planner now has motions it can chain; next we search such chains systematically, starting with the straight drive to the door.

6. Searching the primitives systematically: lattice search

> **Key point:** When every primitive changes the state by whole multiples of a fixed step, all reachable states fall on a regular grid, a lattice, and any graph search can explore it. On the straight drive, breadth-first search finds the fastest safe drive to the door: 7.0 s on a lattice with 0.5 s steps, against 6.2 s for the true best.

Section 3 told us where braking must start; it did not give the fastest drive that stops in front of the door. We search for it with primitives of the straight drive: every 0.5 s the robot accelerates at 0.5 m/s^2, coasts, or brakes at 0.5 m/s^2.

Why the states fall on a grid. In one step of Δt seconds with acceleration u (LaValle §14.4.1.1, eqs. 14.21–14.22):

- s, v, the state at the start of the step;
- u, the acceleration, 0.5, 0 or -0.5 m/s^2;
- Δt, the step, 0.5 s;
- s', v', the state at its end.

$$v' = v + u\,\Delta t$$

$$s' = s + v\,\Delta t + \frac{1}{2} u\,\Delta t^2$$

The speed changes by u times Δt, up or down, or not at all:

$$0.5 \times 0.5 = 0.25 \text{ m/s}$$

so speeds are whole multiples of 0.25 m/s. The position changes by two pieces. The second is half of u times Δt squared:

$$\frac{1}{2} \times 0.5 \times 0.25 = 0.0625$$

The first is the speed v times Δt, with:

- j, the speed counted in steps of 0.25 m/s, a whole number, such as 2 for 0.5 m/s.

$$j \times 0.25 \times 0.5 = j \times 0.125$$

$$= 2j \times 0.0625$$

Both are whole multiples of 0.0625 m, so every position is too. From rest, accelerating once gives:

$$v' = 0 + 0.5 \times 0.5 = 0.25$$

$$s' = 0 + 0 + 0.0625 = 0.0625$$

So every state the robot can reach lies on a grid of 0.0625 m by 0.25 m/s. A regular grid of states like this is a lattice, and searching it for a plan is **lattice search** (G-2827) (LaValle §14.4.1). With speeds of at most 0.6 m/s, only 0, 0.25 and 0.5 m/s fit.

The search. Each 0.5 s step is one move, so the plan with the fewest moves is the fastest. Breadth-first search (G-2550) finds the fewest moves, ring by ring (LaValle §14.4.1.1). A state past the door, s above 3.117 m, or faster than 0.6 m/s, is never added. Figure 7 runs it from rest at s = 0 to the goal, rest at s = 3.0 m. Watch each layer, outlined in orange, add the states reachable in 0.5 s more; the curves between states are the true motions, bent where the speed changes. Notice that states inside the orange region of Section 3 do get added, but none of their moves leads to the goal.

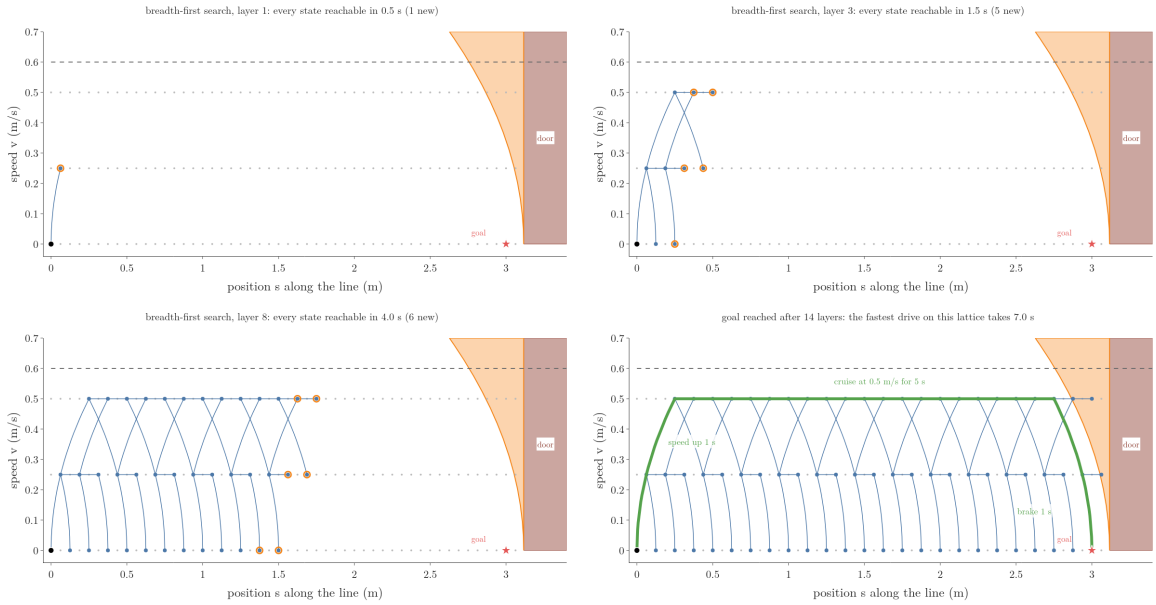


Figure 7: Lattice search on the straight drive, in the phase plane of Figure 3: position s along the line across, speed v up. Faint grey dots: the lattice, positions every 0.0625 m and speeds 0, 0.25 and 0.5 m/s. Breadth-first search starts at rest at s = 0 (black dot); each frame adds one layer of states reachable in 0.5 s more (orange outlines), joined by the true motions (blue curves). Dark red: the door; orange: the region of inevitable collision; red star: the goal, rest at s = 3.0 m. Last frame: after 14 layers the goal is reached; the plan (green) speeds up for 1 s, cruises at 0.5 m/s for 5 s and brakes for 1 s, 7.0 s in all

After 14 layers, 7.0 s, the search reaches the goal, having added 73 states (notebook). The plan is: speed up for two steps to 0.5 m/s, covering 0.25 m; cruise for ten steps, 2.5 m; brake

for two steps, 0.25 m. Braking starts at 2.75 m, before the 2.867 m limit of Section 3 for 0.5 m/s.

How good is 7.0 s? With the true limits, the fastest drive speeds up to 0.6 m/s, cruises, and brakes. Speeding up to 0.6 m/s takes 1.2 s and 0.36 m (Section 2.2), and braking the same. With:

- s_{goal}, the distance to the goal, 3.0 m;
- $v_{\max}$, the top speed, 0.6 m/s;
- a, the acceleration limit, 0.5 m/s^2;
- t_{up}, the time to speed up, and also to brake, in s;
- t_{cruise}, the time at the top speed, in s;
- t, the whole drive, in s.

$$t_{\text{up}} = 0.6/0.5 = 1.2 \text{ s}$$

$$t_{\text{up}} = v_{\max}/a$$

$$t_{\text{cruise}} = (3.0 - 2 \times 0.36)/0.6$$

$$t_{\text{cruise}} = (s_{\text{goal}} - v_{\max}^2/a)/v_{\max}$$

$$t_{\text{cruise}} = 2.28/0.6 = 3.8 \text{ s}$$

$$t = 1.2 + 3.8 + 1.2 = 6.2 \text{ s}$$

$$t = 2\,t_{\text{up}} + t_{\text{cruise}}$$

The lattice could not use 0.6 m/s, because 0.75 m/s, its next speed, is too fast. A finer lattice gets closer (notebook):

Step Δt	Speeds every	Fastest drive	States added
0.5 s	0.25 m/s	7.0 s	73
0.25 s	0.125 m/s	7.0 s	483
0.2 s	0.1 m/s	6.2 s	1043
0.1 s	0.05 m/s	6.2 s	7749

A finer lattice finds better plans but has many more states. If a search at one step finds no plan, halving the step and searching again eventually finds one when one exists (LaValle

§14.4.1.1). The 7.0 s plan, and 6.2 s on a fine enough lattice, answer the stopping part of the opening problem. The turning part needs paths in the plane, and the lattice of Figure 7 has only one direction. We first ask what the best possible path for the turning robot looks like.

7. The shortest drivable paths: Dubins and Reeds-Shepp curves

> **Key point:** For a vehicle that cannot turn tighter than a circle of radius 0.5 m, the shortest path between two poses on an open floor is made of full-lock arcs and straight lines: at most three pieces from a list of six words when it only drives forward (**Dubins path**, G-2828), at most five pieces with two reversals when it may reverse (**Reeds-Shepp path**, G-2829). Both ignore obstacles, so planners use them to connect nearby poses.

7.1 Dubins paths: forward only

The robot at (2, 1) facing 30° at 0.5 m/s must reach the pose in front of the door, (3.6, 3.5) facing 90°, without stopping. What is the shortest path it can drive, if we forget the furniture for a moment?

When our robot keeps moving forward at 0.5 m/s, it moves like a car that drives forward only and cannot turn tighter than a circle of radius 0.5 m. At rest our robot could spin on the spot instead, and its shortest path would then be turn, drive straight, turn (MR §13.3.3); at speed it cannot.

Dubins proved in 1957 that the shortest path is made of at most three pieces, each an arc of the tightest circle or a straight line (MR §13.3.3, Theorem 13.11). Write L for a full left turn, R for a full right turn and S for straight; a path is then a three-letter word (MR §13.3.3). Why only six words?

1. Three letters from L, S and R give 27 words.
2. Two equal letters in a row are one piece, so we drop them, which leaves 12.
3. A word that starts or ends with S cannot be shortest: turning first and then going straight is never longer than going straight and turning later. Dubins's theorem lists only words that turn at both ends (MR §13.3.3, Theorem 13.11). Six words are left: LSL, RSR, LSR, RSL, LRL and RLR.

Building the six candidates. Each pose has a tightest circle on its left and one on its right, both of radius 0.5 m. For LSL, the robot leaves the start's left circle along a line that touches both left circles and joins the goal's left circle. Figure 8 builds the candidates one by one; watch the four circles appear first, then each word with its length.

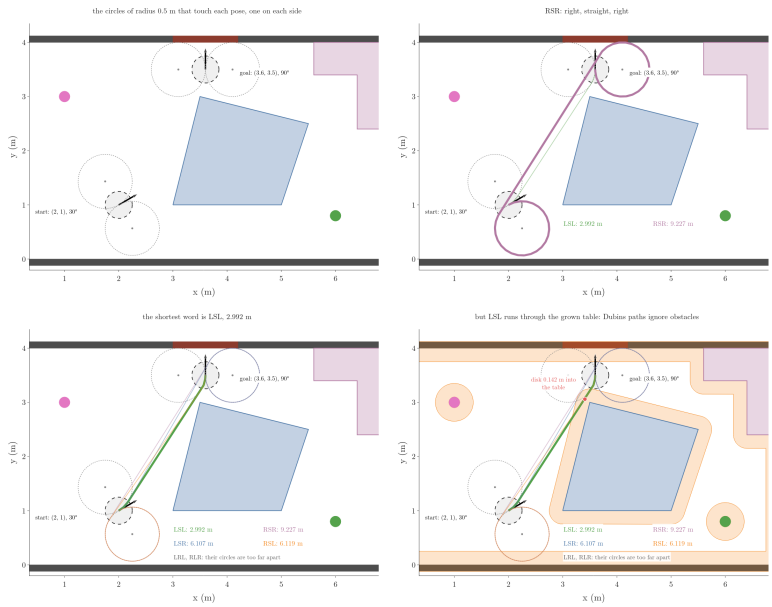


Figure 8: Building the Dubins path from the start (2, 1) facing 30° to the goal (3.6, 3.5) facing 90°, for the tightest circle of radius 0.5 m. First the four circles that touch the poses, one on each side of each. Then each word, highlighted while it is built: LSL 2.992 m, RSR 9.227 m, LSR 6.107 m, RSL 6.119 m; LRL and RLR are impossible because their circles are too far apart. The shortest, LSL, is drawn thick. Last frame: the obstacles grown by the robot's radius, 0.25 m, are added (orange); LSL runs through the grown table, where the disk would overlap the table by up to 0.142 m

The straight piece of LSR is the hardest to see, so we work it out. The start's left circle is centred at (1.75, 1.433) and the goal's right circle at (4.1, 3.5), 3.1297 m apart. The straight piece touches both circles on opposite sides, so the line between the centres is the long side of a right-angled triangle whose short sides are the straight piece and twice the radius (Giese). With:

- D, the distance between the two centres, 3.1297 m;
- ρ, the radius of the tightest circle, 0.5 m;
- ℓ, the length of the straight piece, in m.

$$3.1297^2 = \ell^2 + (2 \times 0.5)^2$$

$$D^2 = \ell^2 + (2\rho)^2$$

$$\ell = \sqrt{9.795 - 1} = 2.966 \text{ m}$$

$$\ell = \sqrt{D^2 - 4\rho^2}$$

which is the 2.966 m straight piece of LSR (notebook). The same triangle shows when a word is impossible: LSR and RSL need the two centres at least 2ρ apart, and LRL and RLR need them at most 4ρ apart, room for a third circle that touches both (Giese). The start's and the goal's left circles are 2.469 m apart, and their right circles 3.468 m, both more than 2 m, so LRL and RLR do not exist here. The shortest word is LSL, 2.992 m: a 0.234 m left arc, 2.469 m straight, a 0.289 m left arc. A path between two poses for a vehicle that drives forward only and turns no tighter than a given radius, shortest among all such paths, is a **Dubins path** (G-2828), also called a Dubins curve (MR §13.3.3).

The last frame of Figure 8 shows the catch: LSL runs through the grown table. A Dubins path knows nothing about obstacles; it is the best drive on an empty floor.

7.2 Reeds-Shepp paths: with reverse

Driving forward only can be very wasteful. Take the sideways move of RO-001: end up 0.5 m to the left of the start, facing the same way, here from (2, 1) facing 30° to (1.75, 1.433) facing 30°. Our robot at rest did it by spin, drive, spin. A car cannot spin, and neither can our robot at speed. Figure 9 compares the two best ways for a vehicle whose tightest circle is 0.5 m. Watch the left robot drive a whole loop. The right robot shuffles: it turns right forward, backs up twice, and turns left forward into place; the red dots mark where it reverses.

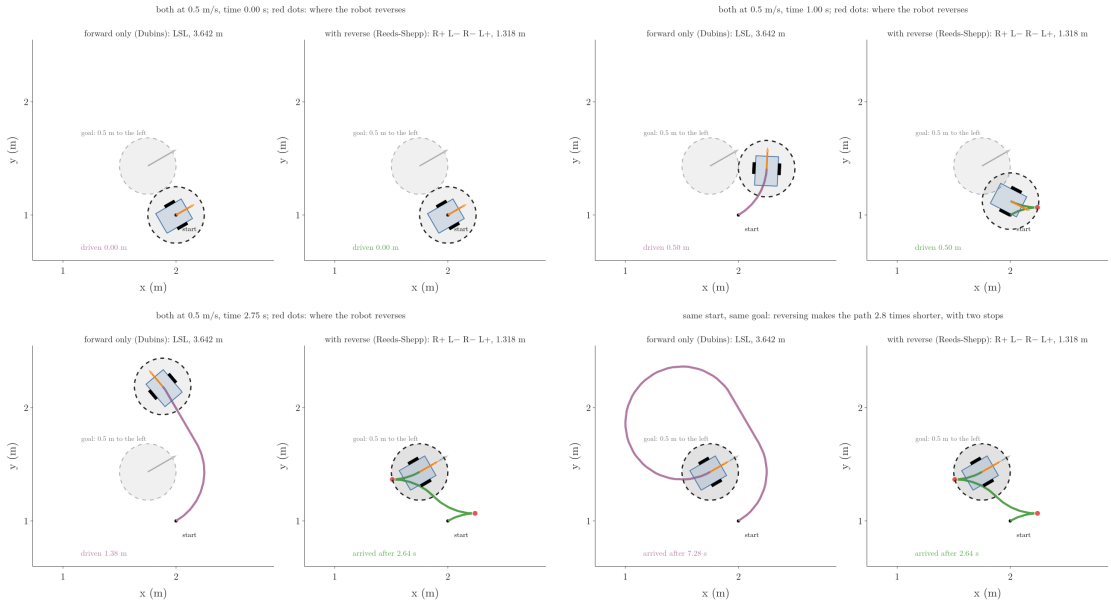


Figure 9: The shortest way to end 0.5 m to the left of the start, (2, 1) facing 30° to (1.75, 1.433) facing 30°, with the tightest circle of radius 0.5 m, both robots at 0.5 m/s, one frame per 0.25 s. Left, forward only (Dubins): LSL, a quarter turn left, 0.5 m straight, three quarters of a turn left, 3.642 m, arriving after 7.28 s. Right, with reverse (Reeds-Shepp): R+ L− R− L+ (+ forward, − reverse), 1.318 m with 2 reversals (red dots), arriving after 2.64 s of driving. Last frame: the path with reverse is 2.8 times shorter

With reverse the path is 1.318 m against 3.642 m, 2.8 times shorter, at the price of two stops

to change direction (notebook). Reeds and Shepp proved in 1990 that, for a car that may also reverse, the shortest path between two poses again uses only arcs of the tightest circle and straight lines, now with some pieces driven backwards. Every shortest path falls into one of nine families with at most five pieces and at most two reversals; with L or R driven forward (+) or in reverse (−), they make 48 words (MR §13.3.3, Theorem 13.12). The shortest path from this list is the **Reeds-Shepp path** (G-2829), also called a Reeds-Shepp curve. Our sideways move is R+ L− R− L+: 0.253 m forward right, 0.406 m backward left, 0.406 m backward right, 0.253 m forward left. Driving these four pieces in turn from the start ends exactly on the goal pose, which the notebook checks for every candidate path.

Two consequences matter for planning. First, with reverse a car can get to any pose near it with a short path, sideways included, by shuffling as in parallel parking; forward only, a pose just behind it costs a whole loop: 0.5 m straight back is 0.5 m with reverse and 3.642 m without (MR §13.3.2; notebook). Second, the distance with metres and radians (G-2681) of RO-038 calls our sideways goal 0.5 m away, although the shortest drive is 1.318 m with reverse and 3.642 m without. Distances that ignore the motion limits can badly misjudge which pose is near (MR §10.5; Hauser ch.11). Section 8 meets this problem again.

7.3 Using them in a planner: a drivable steer

Dubins and Reeds-Shepp paths are exact but blind to obstacles; planners are aware of obstacles but need drivable pieces. The two meet in the planner's steer (G-2763), the step that moves from one configuration towards another, in RO-051 along a straight line. For a car the steer must return a drivable path that ends exactly on the target pose, or report that it cannot; such a steer is a local planner (G-2782), the fast method that tries to join two configurations with one simple motion (Hauser ch.11; MR §10.5). The Reeds-Shepp path is a natural choice (Hauser ch.11; MR §10.5), and Hybrid A*'s shot to the goal uses it to reach a parking pose exactly.

One way to use it turns any free route of straight pieces into a drivable path (MR §13.3.3). Take a route that is free but has corners, such as A*'s: here three straight pieces from the start round the table's left side to the pose in front of the door. Join its two ends with a Reeds-Shepp path. If that path hits an obstacle, cut the route at its middle, take the route's pose there (its point, facing along the route), and join each half the same way. Repeat on every half that fails. Figure 10 runs it; watch the red tries fail and the green ones stay.

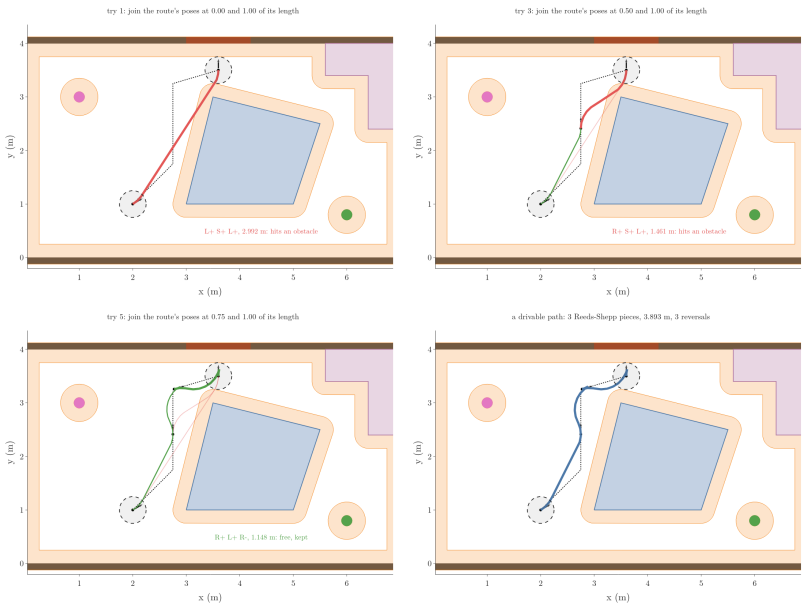


Figure 10: Turning a free route of three straight pieces (black, dotted) into a drivable path with Reeds-Shepp pieces for the tightest circle of 0.5 m. Each try joins two poses of the route (black arrows): red when the path hits an obstacle grown by 0.25 m (orange), green when it is free. Try 1, start to goal, L+ S+ L+ 2.992 m, hits the grown table; try 2, the first half, is free; try 3, the second half, hits; tries 4 and 5, the two quarters, are free. Last frame: the drivable path (blue), 3 pieces, 3.893 m, with 3 reversals

Five tries give a drivable path of three pieces, 3.893 m, with 3 reversals (notebook). Halving always ends for a car that can reverse, because the route has some free space around it, the car can follow any path as closely as we like by shuffling, and the Reeds-Shepp paths between ever closer poses become ever shorter, so they stay near the route (MR §13.3.3). The result is drivable, but its three reversals mean three stops, and this method needs reverse. For the robot that keeps moving forward we need a planner that builds paths out of the forward primitives themselves.

8. Growing a tree of drivable motions: kinodynamic RRT

> **Key point:** The kinodynamic RRT grows a tree whose every edge is a primitive run through the simulator: it samples a state, takes the nearest tree node, tries the primitives from it and keeps the one that comes closest, if its motion is free. Every path in the tree is drivable by construction; in the room it reaches the goal region in front of the door with 8 primitives, 3.0 m, at 0.5 m/s without a stop.

The RRT grows a tree from the start towards random samples, each new edge a straight step towards the sample. For our moving robot a straight edge is the problem we started with: it may ask for a sideways move or a turn on the spot. The fix keeps the RRT's loop and changes only how an edge is made (MR §10.5; LaValle 1998):

1. Sample a state at random; one time in ten, take the goal pose instead, a goal bias (G-2764) of 0.1, to pull the tree towards it.
2. Find the nearest node (G-2762) to the sample, by RO-038's distance, which counts 0.25 m for each radian of turn.
3. In place of the straight steer, run each of the three primitives of Section 5 from that node through the simulator, and keep the one whose end comes closest to the sample.
4. If its whole motion stays in free space, add its end as a new node, with the primitive as the edge; otherwise throw it away.
5. Stop when a new node lands in the goal region.

An RRT whose edges are simulated motions is a **kinodynamic RRT** (G-2832) (LaValle §14.4.3; Hauser ch.11). Figure 11 runs it in the room, with the obstacles grown by the robot's radius. Watch the first frames: the red cross is the sample, the black dot the nearest node, the grey arcs the three primitives tried, the thick one the closest, green if kept and red if thrown away. Then the frames jump 15 iterations at a time.

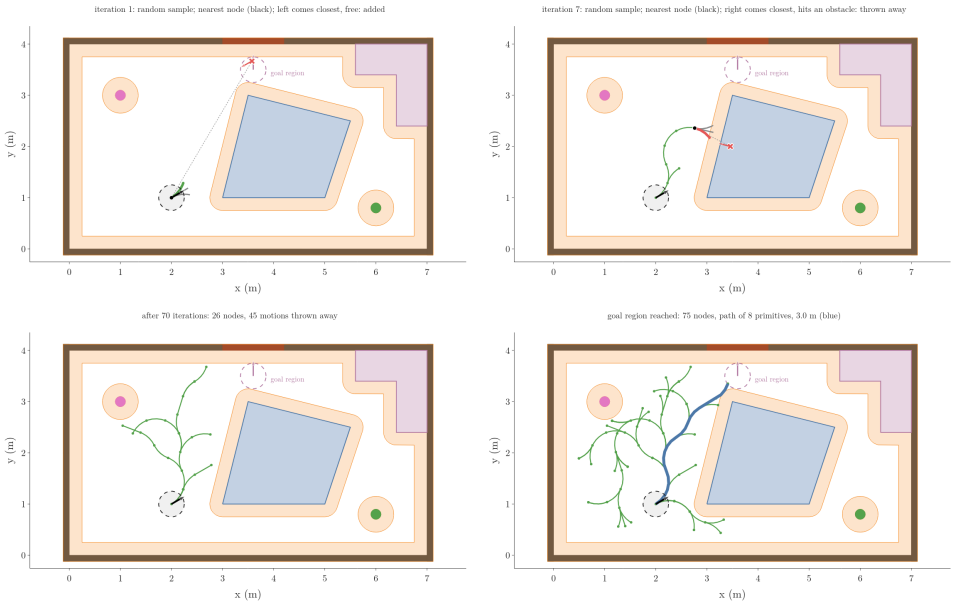


Figure 11: The kinodynamic RRT in the room (seed 4), from (2, 1) facing 30° at 0.5 m/s, obstacles grown by 0.25 m (orange). In each of the first 12 frames: the sample (red cross and arrow), the nearest tree node (black dot), the three primitives tried from it (grey), and the one that ends closest to the sample, green when free and added, red when it would hit an obstacle and is thrown away. Later frames jump 15 iterations. The goal region (purple dashed circle): within 0.25 m and 25° of (3.6, 3.5) facing 90°. Last frame: after 221 iterations, 147 of them thrown away, the tree has 75 nodes and the path to the goal region (blue) is 8 primitives, 3.0 m

After 221 iterations, 147 of them thrown away, the tree has 75 nodes and a node lies in the goal region: 0.243 m from (3.6, 3.5), facing 73° (notebook). Its path is 8 primitives, 3.0 m: left, left, right, right, left, right, straight, left. Each is a 0.5 m/s motion with a turn rate of

at most 1 rad/s, so the robot drives it without a stop, and it passes left of the table where the Dubins path of Figure 8 went through it. The run shown is typical: over 20 seeds, 18 reached the goal region within 4000 nodes, with a median of 76.5 nodes and a median path of 3.375 m (notebook).

Three details explain the method's behaviour:

- **A goal region, not a goal pose.** Chains of fixed primitives almost never land exactly on one pose, so the tree aims at a small region around it (Hauser ch.11). A drivable steer, such as the Reeds-Shepp path of Section 7.3, can then join the last node to the exact pose.
- **Many throw-aways.** Choosing the primitive that comes closest often points it into an obstacle; 147 of 221 tries were thrown away. The Modern Robotics book keeps instead the closest free one (MR §10.5.1.3).
- **The distance misleads.** RO-038's distance prefers a node beside the sample, which, as Section 7.2 showed, may be a whole loop away for a robot that cannot reverse. A distance closer to the true driving distance makes the tree grow better, and finding one is a known difficulty of kinodynamic RRT (MR §10.5; Hauser ch.11).

The kinodynamic RRT answers the turning part of the opening question: it plans in the room with only motions the robot can drive at speed. Its path is not the shortest, and random sampling gives a different path each run.

9. Summary

Idea	What it is	On our drive to the door
Phase space	configurations together with their speeds	the state of the straight drive is (s, v)
Kinodynamic planning	planning with obstacles and with limits on how the state can change	inputs between -0.5 and 0.5 m/s^2, speeds up to 0.6 m/s
Region of inevitable collision	free states from which every input leads into an obstacle	at 0.6 m/s, every s past 2.757 m
Reachable set	every state reachable within a time limit	at 0.5 m/s, a fan between the 0.5 m circles
Motion primitive, system simulator	a short drivable motion; the module that steps the motion equations	left, straight, right for 0.75 s, 0.01 s Euler steps
Lattice search	graph search over states that primitives keep on a grid	fastest safe drive: 7.0 s (6.2 s at best)
Dubins and Reeds-Shepp paths	shortest paths for a tightest circle, forward only or with reverse	2.992 m to the door pose; sideways: 3.642 m against 1.318 m
Drivable steer	joins two states with a drivable path, or reports that it cannot	Reeds-Shepp pieces: 3.893 m with 3 reversals
Kinodynamic RRT	an RRT whose edges are simulated primitives	8 primitives, 3.0 m, no stops

- The opening question has its answer. Stopping: from 0.6 m/s the robot needs 0.36 m, so it must brake by s = 2.757 m, and the fastest drive that stops in front of the door takes 7.0 s on the coarse lattice and 6.2 s at best. Turning: the kinodynamic RRT reaches the goal region in front of the door with 8 primitives, 3.0 m, all at 0.5 m/s, because every edge is a motion the robot can drive.
- Speed is part of the state because the same position can be safe or lost: at 2.8 m, 0.3 m/s stops in time and 0.6 m/s cannot, so a planner with motion limits works in **phase space** (G-2820) and does **kinodynamic planning** (G-2821).
- The door forbids every state from 3.117 m on, the **obstacle region in phase space** (G-2822), and the **region of inevitable collision** (G-2823) adds every state whose stopping distance is longer than the room left, which grows with the square of the speed because the stopping distance is v^2 over 2a.
- At 0.5 m/s the robot's **reachable set** (G-2824) is a fan between its two tightest circles, so a turn on the spot is not among its moves and any plan must be built from motions inside such fans.
- A few **motion primitives** (G-2826), run through the **system simulator** (G-2825), give drivable building blocks, because each one holds an allowed input for a fixed time.
- When the primitives change the state by whole steps, the states form a lattice and

lattice search (G-2827) with breadth-first search finds the fastest plan on it, because each move takes the same time; finer lattices come closer to the true best.
- The **Dubins path** (G-2828) and the **Reeds-Shepp path** (G-2829) are the shortest drives between two poses for a tightest circle, so they make good drivable steers; they ignore obstacles, and each reversal adds a stop.
- The **kinodynamic RRT** (G-2832) keeps the RRT's sample, nearest node and extend loop but extends with simulated primitives, so every path it returns is drivable.

The robot can now plan drives it can follow at speed, in a room. Still missing: a car cannot spin even at rest, and parking it among other cars needs a search that is both systematic and drivable, with headings in its states and arcs as its edges, which is Hybrid A*.

10. Sources

Built from

- Northwestern Robotics, "Modern Robotics, Chapter 10.5: Sampling Methods for Motion Planning (Part 2 of 2)", YouTube, https://www.youtube.com/watch?v=Ao7p_xiUu4s
- Northwestern Robotics, "Modern Robotics, Chapter 13.3.3: Motion Planning for Nonholonomic Mobile Robots", YouTube, https://www.youtube.com/watch?v=jOesC0wKpTQ
- Aaron Becker, "Coding a Dubins Car Optimal Path Planner", YouTube, https://www.youtube.com/watch?v=tjUVE3jto98
- Northwestern Robotics, "Modern Robotics, Chapter 13.3.2: Controllability of Wheeled Mobile Robots (Part 2 of 4)", YouTube, https://www.youtube.com/watch?v=S7Y2aUQiDCw
- Hauser, K. *Robotic Systems*, ch. 11 "Planning with Dynamics and Uncertainty": Kinodynamic planning, Path planning with curvature constraints, Local planning with a steering function, Kinodynamic sampling-based planning. Free online: https://motion.cs.illinois.edu/RoboticSystems/PlanningWithDynamicsAndUncertainty.html (Hauser ch.11)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §14.1.2.1 "Terms from planning", §14.1.3 "Obstacles in the phase space" (with Example 14.1), §14.2.1 "Reachable sets", §14.2.2 "The discrete-time model", §14.2.3 "Motion primitives", §14.3.2 "System simulator", §14.4.1 "Searching on a lattice", §14.4.3 "RDT-based methods". Free online: https://lavalle.pl/planning/node726.html (LaValle)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.5 (RRT, its local planners), §13.3.1 (car kinematics, control sets, Figure 13.11), §13.3.2 (controllability, reachable sets), §13.3.3 (Dubins and Reeds-Shepp paths, planning among obstacles). Free preprint: http://modernrobotics.org (MR)

Other references

- Dubins, L. E. (1957). On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents. *American Journal of Mathematics* 79(3), 497–516.
- Reeds, J. A. and Shepp, L. A. (1990). Optimal paths for a car that goes both forwards and backwards. *Pacific Journal of Mathematics* 145(2), 367–393.

- LaValle, S. M. (1998). Rapidly-exploring random trees: a new tool for path planning. Technical report TR 98-11, Iowa State University.
- Giese, A. "A Comprehensive, Step-by-Step Tutorial to Computing Dubin's Paths", https://gieseanw.wordpress.com/2012/10/21/a-comprehensive-step-by-step-tutorial-to-computing-dubins-paths/ (Giese)
- The Reeds-Shepp formulas in the notebook follow the open-source OMPL library's `ReedsSheppStateSpace`, https://ompl.kavrakilab.org; every candidate path is checked by driving it to the goal.

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Phase space (G-2820)	The space of a robot's configurations together with their speeds, such as the pairs (position, speed) of a robot driving along a line; planning there lets a planner tell a robot that can still stop from one that cannot.
Kinodynamic planning (G-2821)	Planning a motion that avoids obstacles and also obeys limits on how the state can change, $\dot{x} = f(x, u)$ with bounded inputs, such as a top speed and an acceleration limit; its plans can be driven as they are, without stops the robot cannot make.
Region of inevitable collision (inevitable collision states) (G-2823)	The free states from which every allowed input sooner or later leads into an obstacle, such as a robot at 0.6 m/s with only 0.317 m left before a wall when it needs 0.36 m to stop; it grows with the square of the speed, so fast robots must keep well out of it.
Obstacle region in phase space (G-2822)	Every state whose configuration touches an obstacle, at any speed, such as every (position, speed) from 3.117 m on in front of a closed door; a kinodynamic plan must stay out of it.
Reachable set (G-2824)	Every state a robot can reach from a given state within a time limit using allowed inputs, such as the fan of positions in front of a robot driving at 0.5 m/s that turns at most 1 rad/s; it shows which moves a plan can use from there.

Term	Meaning
Motion primitive (G-2826)	A short motion the robot can drive, usually one input held for a fixed time, such as full left at 0.5 m/s for 0.75 s; planners chain primitives so that every path they build is drivable.
System simulator (G-2825)	The module that works out where an input takes the robot, by stepping its motion equations forward, such as 75 Euler steps of 0.01 s; planners call it to turn inputs into motions they can check for collisions.
Lattice search (G-2827)	Graph search over states that the primitives keep on a regular grid, such as positions every 0.0625 m and speeds every 0.25 m/s when the robot accelerates, coasts or brakes every 0.5 s; any graph search then finds the best plan on that grid.
Dubins path (Dubins curve) (G-2828)	The shortest path between two poses for a vehicle that drives forward only and turns no tighter than a given radius: at most three pieces, full-lock arcs and a straight line, one of the words LSL, RSR, LSR, RSL, LRL, RLR; it is the best forward drive on an empty floor.
Reeds-Shepp path (Reeds-Shepp curve) (G-2829)	The shortest path between two poses for a vehicle that turns no tighter than a given radius but may also reverse: at most five full-lock arcs and straight pieces with at most two reversals, from 48 words in nine families; planners use it to join two poses exactly with a drivable path.
Kinodynamic RRT (G-2832)	A rapidly-exploring random tree whose edges are motion primitives run through the system simulator: it samples a state, takes the nearest node, keeps the primitive that ends closest if its motion is free; every path it returns is drivable.
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).

Term	Meaning
Rapidly-exploring random tree (RRT) (G-2761)	A sampling-based planner that grows a tree from the start: each round it draws a random configuration, steps the tree's nearest node at most a fixed distance towards it and keeps the new node if the step is free, until the tree reaches the goal; it explores fast but its routes zig-zag.
Inverse kinematics (wheeled robot) (G-2295)	Working out the wheel speeds that give a wanted linear and angular velocity, used by a controller to command the motors.
Linear velocity v (G-2291)	How fast a robot's reference point (for a differential drive, the middle of the axle) moves along its heading, in m/s; for a differential drive, the average of the two wheel speeds, so it sets how fast the robot advances.
Angular velocity ω (G-2292)	How fast a robot's heading changes, in rad/s, positive to the left; for a differential drive, the difference of the wheel speeds divided by the wheel separation.
Turning radius R (G-2293)	The distance from the point a robot is turning about (the instantaneous centre of curvature, ICC) to the robot's reference point; $R = v/\omega$, so it says how tight a turn is.
Heading θ (G-2284)	The angle from the world x-axis to the direction a robot faces, measured counter-clockwise; it decides which way the robot moves when it drives forward.
Dot notation $\dot{x}$ (G-2296)	A dot over a quantity means its rate of change per second (its derivative with respect to time), such as $\dot{x} = 0.433$ m/s; models of motion are written with it.
Configuration space (C-space) (G-2287)	The set of all configurations of a robot, one point per placement of the whole robot, such as a box of (x, y, θ) whose top and bottom faces are the same headings for a floor robot; motion planning looks for paths through it.

Term	Meaning
State space (G-2545)	All the states a problem can be in, joined by the moves between them, such as the robot's 76 cells or the arrangements of a Rubik's cube; it is a graph, so planning becomes finding a path from the start state to a goal state.
Obstacle region (C-obstacle) (G-2650)	The set of all configurations at which the robot's body touches or overlaps an obstacle, C_{obs}; drawing it in C-space lets a planner test a placement as one point, such as the disk robot's centre (3.5, 3.2), which lies in it 0.2 m from the table.
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.
Kinematic model (G-2297)	Equations that give how a robot's configuration changes from its speed inputs alone, ignoring masses and forces, such as $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = \omega$; used to predict motion at low speed.
Breadth-first search (BFS) (G-2550)	A search whose frontier is a queue, so it expands vertices in rings of equal moves from the start; the first route it finds has the fewest moves, which is the shortest only when every move costs the same.
Metric (distance function) (G-2681)	A distance between points that is never negative, is 0 only between equal points, is the same both ways and obeys the triangle inequality; search trees rely on these rules to skip far regions safely.
Steer (steering function) (G-2763)	The step of a sampling-based planner that moves from a tree node towards a sample, at most a fixed distance (0.5 m here), stopping on the sample if it is closer; it keeps every new edge short enough to check.
Local planner (G-2782)	The fast method a sampling planner uses to try to join two configurations, usually the straight segment; it proposes one simple motion without searching round obstacles, and a collision check reports whether that motion is free.

Term	Meaning
Goal bias (G-2764)	The small probability (0.05 here) with which a sampling-based planner uses the goal itself as its next sample, so the tree keeps trying to reach the goal; too large a bias makes it greedy, too small leaves it wandering.
Nearest node (G-2762)	The node of a planner's tree closest to a new sample under the planner's distance; RRT extends it, which makes the tree grow towards the largest empty regions.